

ASSIGNMENT : 12

ADVANCED PROGRAMMING

NAME:BHITALI BORUAH

ROLLNO:CSB24045

SESSION:SPRING

YEAR:2024-28

```
1 import java.util.Scanner;
2
3 interface PaymentMethod {
4     void pay(double amount);
5 }
6
7 interface NotificationService {
8     void send(String message);
9 }
10
11 interface OrderRepository {
12     void save(Order order);
13 }
14
15 class CreditCardPayment implements PaymentMethod {
16     public void pay(double amount) {
17         System.out.println("Paid using Credit Card: " + amount);
18     }
19 }
20
21 class UpiPayment implements PaymentMethod {
22     public void pay(double amount) {
23         System.out.println("Paid using UPI: " + amount);
24     }
25 }
26
27 class WalletPayment implements PaymentMethod {
28     public void pay(double amount) {
29         System.out.println("Paid using Wallet: " + amount);
30     }
31 }
32
33 class EmailNotification implements NotificationService {
34     public void send(String message) {
35         System.out.println("Email: " + message);
36     }
37 }
```

```
33 class EmailNotification implements NotificationService {
34     }
35 }
36
37 class SMSNotification implements NotificationService {
38     public void send(String message) {
39         System.out.println("SMS: " + message);
40     }
41 }
42
43 class PushNotification implements NotificationService {
44     public void send(String message) {
45         System.out.println("Push Notification: " + message);
46     }
47 }
48
49 class DatabaseStorage implements OrderRepository {
50     public void save(Order order) {
51         System.out.println("Saved to Database: " + order);
52     }
53 }
54
55 class FileStorage implements OrderRepository {
56     public void save(Order order) {
57         System.out.println("Saved to File: " + order);
58     }
59 }
60
61 class Order {
62     protected double amount;
63
64     public Order(double amount) {
65         this.amount = amount;
66     }
67
68     public double getAmount() {
69         return amount;
70     }
71 }
```

This screenshot shows the VS Code editor with the Explorer view on the left displaying a project structure. The main editor window shows the implementation of the `Order` and `DiscountedOrder` classes. The `Order` class has methods `getAmount()` and `toString()`. The `DiscountedOrder` class extends `Order` and implements `getAmount()` with a 10% discount. The `OrderService` class is also partially visible, containing private fields for `payment`, `notification`, and `repository`, and a constructor.

```
63 class Order {
64     public double getAmount() {
65         return amount;
66     }
67
68     public String toString() {
69         return "Order Amount: " + amount;
70     }
71 }
72
73 class DiscountedOrder extends Order {
74     public DiscountedOrder(double amount) {
75         super(amount * 0.9);
76     }
77 }
78
79 class PriorityOrder extends Order {
80     public PriorityOrder(double amount) {
81         super(amount + 50);
82     }
83 }
84
85 class OrderService {
86     private PaymentMethod payment;
87     private NotificationService notification;
88     private OrderRepository repository;
89
90     public OrderService(PaymentMethod payment,
91                         NotificationService notification,
92                         OrderRepository repository) {
93         this.payment = payment;
94         this.notification = notification;
95         this.repository = repository;
96     }
97
98     public void placeOrder(Order order) {
99         payment.pay(order.getAmount());
100     }
101 }
```

This screenshot shows the VS Code editor with the Explorer view on the left displaying a project structure. The main editor window shows the implementation of the `Main` class and the `OrderService` class. The `OrderService` class has a `placeOrder()` method that uses the `payment`, `notification`, and `repository` objects. The `Main` class contains a `main` method that uses a `Scanner` to get user input for the order amount, order type, and payment method, and then calls the `placeOrder` method of the `OrderService` class.

```
91 class OrderService {
92     public void placeOrder(Order order) {
93         payment.pay(order.getAmount());
94         repository.save(order);
95         notification.send(message: "Order placed successfully!");
96     }
97 }
98
99 public class Main {
100     Run main | Debug main | Run | Debug
101     public static void main(String[] args) {
102
103         Scanner sc = new Scanner(System.in);
104
105         System.out.println(x: "Enter Order Amount:");
106         double amount = sc.nextDouble();
107
108         System.out.println(x: "Select Order Type: 1.Regular 2.Discounted 3.Priority");
109         int orderChoice = sc.nextInt();
110
111         Order order;
112         if (orderChoice == 1) {
113             order = new Order(amount);
114         } else if (orderChoice == 2) {
115             order = new DiscountedOrder(amount);
116         } else if (orderChoice == 3) {
117             order = new PriorityOrder(amount);
118         } else {
119             order = new Order(amount);
120         }
121
122         System.out.println(x: "Select Payment Method: 1.Credit Card 2.UPI 3.Wallet");
123         int payChoice = sc.nextInt();
124
125         PaymentMethod payment;
126         if (payChoice == 1) {
127             payment = new CreditCardPayment();
128         } else if (payChoice == 2) {
129             payment = new UpiPayment();
130         }
131
132         OrderService orderService = new OrderService(payment, notification, repository);
133         orderService.placeOrder(order);
134     }
135 }
```

```
111 public class Main {
112     public static void main(String[] args) {
113         payment = new UpiPayment();
114     } else {
115         payment = new WalletPayment();
116     }
117
118     System.out.println(x: "Select Notification: 1.Email 2.SMS 3.Push");
119     int notifchoice = sc.nextInt();
120
121     NotificationService notification;
122     if (notifchoice == 1) {
123         notification = new EmailNotification();
124     } else if (notifchoice == 2) {
125         notification = new SMSNotification();
126     } else {
127         notification = new PushNotification();
128     }
129
130     System.out.println(x: "Select Storage: 1.Database 2.File");
131     int storechoice = sc.nextInt();
132
133     OrderRepository storage;
134     if (storechoice == 1) {
135         storage = new DatabaseStorage();
136     } else {
137         storage = new FileStorage();
138     }
139
140     OrderService service = new OrderService(payment, notification, storage);
141
142     service.placeOrder(order);
143 }
144 }
```

OUTPUT:

```
c:\Users\VTU\OneDrive\Desktop\java>cd "c:\Users\VTU\OneDrive\Desktop\java" && javac Main.java && java Main
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.encoding=UTF-8
Enter Order Amount:
1000
Select Order Type: 1.Regular 2.Discounted 3.Priority
2
Select Payment Method: 1.Credit Card 2.UPI 3.Wallet
2
Select Notification: 1.Email 2.SMS 3.Push
1
Select Storage: 1.Database 2.File
1
Paid using UPI: 900.0
Saved to Database: Order Amount: 900.0
Email: Order placed successfully!

c:\Users\VTU\OneDrive\Desktop\java>
```